

SQL поддерживает набор инструкций, предназначенных для изменения данных в таблицах. Это подмножество команд языка называется **Data Manipulation Language, DML**.

DML содержит команды для вставки записей в таблицы, модификации существующих записей, а также удаления данных.

1. Вставка данных.

Для добавления записей в таблицу используется команда **INSERT**. Существует два варианта этой команды:

- с использованием ключевого слова **VALUES** - для вставки одной записи
- с использованием оператора **SELECT** - для вставки набора записей

Синтаксис:

```
INSERT [INTO] [database.[owner.]]{tableName | viewName}
  [(columnName [, columnName ...])]
  { VALUES ( constantExpr [, constantExpr ...] )
  | SELECT selectDefinition }
```

Здесь **SELECT selectDefinition** - это оператор **SELECT**, содержащий любые свои разделы, кроме **COMPUTE**, при помощи которого формируется набор данных, добавляемый в целевую таблицу.

Вне зависимости от применяемого варианта инструкции **INSERT**, сразу после имени таблицы идет необязательный блок, в котором в скобках перечисляются столбцы таблицы, в которую добавляются данные. Перечень столбцов можно не указывать, но тогда количество и порядок значений в разделе **VALUES** или полей в списке выбора **SELECT** должны совпадать с количеством и порядком следования полей целевой таблицы, указанным при ее создании (в инструкции **CREATE TABLE**).

Например,

```
CREATE TABLE myTable( code int NOT NULL,
                      description varchar(100) DEFAULT '' NULL )

INSERT INTO myTable VALUES( 1, 'XXX' )
```

Однако использовать такой подход на практике не рекомендуется, поскольку инструкция **INSERT** становится зависимой от порядка и количества полей в целевой таблице. Кроме того, так будет сложнее отыскать возможные ошибки.

Правильнее четко перечислить поля целевой таблицы. В этом случае перечень значений в разделе **VALUES** или список выбора оператора **SELECT** будет определяться порядком перечисления полей в блоке **INSERT**. При этом в блоке **INSERT** поля могут быть перечислены в любом порядке.

```
INSERT INTO myTable( description, code ) VALUES( 'XXX', 1 )
```

Если какие-либо поля целевой таблицы не указаны в перечне полей **INSERT**, эти поля заполняются

значением по умолчанию (естественно, если оно определено), либо NULL.

Если поле не указано в перечне полей INSERT и не допускает значений NULL, а также не имеет значения по умолчанию, при выполнении инструкции INSERT возникнет ошибка.

Если для поля описано значение по умолчанию, но в это поле явно вставляется NULL, будет записан NULL (конечно, если поле это допускает).

Пример.

```
-- В поле description запишется '' - значение по умолчанию
INSERT INTO myTable( code ) VALUES( 2 )

-- В поле description запишется NULL
INSERT INTO myTable( code, description ) VALUES( 3, NULL )

-- Ошибка: поле code не допускает NULL и не имеет значения по умолчанию
INSERT INTO myTable( description ) VALUES( 'YYY' )
```

Использование формы с SELECT:

```
/*
 * записываем в myTable информацию обо всех клиентах,
 * родившихся после 1.01.1980
 */
INSERT INTO myTable( code, description )
SELECT ClientID, LastName + ' ' + FirstName
FROM Clients
WHERE DateBorn > '1980-01-01'

-- Добавление одной строки при помощи SELECT
INSERT INTO myTable( code, description ) SELECT 4, 'ZZZ'
```

Если таблица содержит поля **timestamp** или **identity**, перечисление столбцов таблицы в разделе **INSERT** становится единственным возможным способом добавления записей. При этом поле **timestamp** или **identity** в списке полей не перечисляется и значение для него не указывается. Оно будет сгенерировано автоматически.

```
CREATE TABLE myTable( code int IDENTITY,
                        description varchar(100),
                        stamp timestamp )

-- Значения code и stamp будут сгенерированы автоматически
INSERT INTO myTable( description ) SELECT 'AAA'
```

Иногда возникает необходимость задать значение поля **identity** явно. Например, запись была ошибочно удалена и ее нужно восстановить.

В такой ситуации владелец таблицы, а также владелец базы данных или администратор системы (роль **sa**) при условии, что они действуют от лица владельца таблицы (команда **setuser**), могут

перед вставкой данных установить опцию **identity_insert** для данной таблицы.

В рамках одной сессии в каждый момент времени можно устанавливать опцию **identity_insert** только для одной таблицы. После включения опции **identity_insert**, имя поля **identity** должно быть явно указано в списке столбцов, а значение для него в списке **VALUES**.

ASE не отслеживает уникальность явно вставляемых данных. После отключения опции автоматическая генерация продолжается, начиная со значения, следующего за вставленной константой.

Пример.

```
set identity_insert on
insert into myTable( code, description ) values( 25, 'BBB' )
set identity_insert off
```

Для того чтобы узнать последнее вставленное в поле **identity** значение, можно использовать глобальную переменную **@@identity**.

Пример.

```
CREATE TABLE list( code int PRIMARY KEY IDENTITY,
                   description varchar(100) )
go

CREATE TABLE addData( code int,
                      subCode int,
                      data varchar(255) NULL,
                      CONSTRAINT addDataPK PRIMARY KEY ( code, subCode ),
                      FOREIGN KEY( code ) REFERENCES list( code ) )
go

declare @code int

-- добавляем запись в основную таблицу
INSERT INTO list( description ) VALUES 'TTT'

-- фиксируем последнее сгенерированное значение поля identity
select @code = @@identity

-- добавляем запись в связанную таблицу
INSERT INTO addData( code, subCode, data ) SELECT @code, 1, 'Add data'
go
```

2. Изменение данных в таблице.

Для изменения данных в таблице применяется инструкция **UPDATE**:

```
UPDATE tableName
SET { variable | columnName } = expression [, columnName = expression]
[FROM { tableName | vewName } [, { tableName | vewName } ...]]
[WHERE searchCondition]
```

Здесь **expression** - имя поля, константа, выражение или подзапрос.

Как и другие операции **DML**, одна команда **UPDATE** может изменять поля только одной таблицы. При помощи команды **UPDATE** можно изменять как значения полей таблицы, так и значения переменных.

Команда **UPDATE** изменяет все строки целевой таблицы, удовлетворяющие условиям поиска. Таким образом, одной командой можно выполнить изменение целого набора записей.

Условия отбора записей описываются в блоке **WHERE** аналогично оператору **SELECT**. Для гарантированного изменения одной записи таблицы следует проводить поиск по первичному ключу. Если предложение **WHERE** не используется, изменяются все записи в таблице.

Базовый вариант UPDATE:

```
-- Принудительно делаем первую букву фамилии и имени заглавной,
-- остальные - строчными. Для всех, у кого фамилия начинается на 'А'
UPDATE Clients
SET LastName = upper( substring( LastName , 1, 1 ) ) +
               lower( substring( LastName , 2 ) ),
    FirstName = upper( substring( FirstName , 1, 1 ) ) +
               lower( substring( FirstName , 2 ) )
WHERE LastName like 'А%'
```

Предложение FROM.

Предложение **FROM** в операторе **UPDATE** используется, если нужно изменить информацию в одной таблице на основании информации из другой.

Логику выполнения запроса в этом случае можно разбить на два этапа:

- на первом этапе, в результате выполнения оператора **SELECT**, включающего объединение таблиц, строится набор строк
- на втором, на основе полученного набора выполняется команда **UPDATE**

Альтернативным использованию предложения **FROM** в команде **UPDATE** вариантом может служить команда **UPDATE**, использующая подзапрос в предложении **SET**. Причем этот вариант является ANSI-совместимым.

Если обычное объединение в Transact SQL может включать до **16** таблиц, то объединение, являющееся частью команды **UPDATE** или **DELETE**, может состоять не больше, чем из **15** таблиц.

При необходимости выполнить связку с изменяемой таблицей, ее наименование также помещается в раздел **FROM**.

Примеры.

```
-- Добавляем в таблицу клиентов поле-флаг наличия счета (Y/N)
ALTER TABLE Clients ADD HasAccount char(1) default 'N'

-- Выставляем значение этого флага в 'Y' для клиентов, имеющих счета
UPDATE Clients
SET HasAccount = 'Y'
FROM Clients c, Accounts a
WHERE a.ClientID = c.ClientID

-- То же самое через EXISTS
UPDATE Clients
SET HasAccount = 'Y'
FROM Clients c
WHERE EXISTS( SELECT 1 FROM Accounts a WHERE a.ClientID = c.ClientID )

-- То же самое с использованием подзапроса
UPDATE Clients
SET HasAccount = isnull( ( SELECT DISTINCT 'Y'
                           FROM Accounts a
                           WHERE a.ClientID = c.ClientID ),
                        'N' )
FROM Clients c
```

Использование переменных.

Как уже говорилось, основное назначение инструкции **UPDATE** - это изменение информации в таблицах. Однако в **Transact SQL** слева от знака равенства в предложении SET могут находиться переменные.

Во-первых, поскольку оператор **UPDATE** имеет итерационную природу, это позволяет выполнять с помощью **UPDATE** циклические операции:

```
DECLARE @cnt int

SELECT @cnt = 1

UPDATE SomeTable
SET RecNumber = @cnt,
    @cnt = @cnt + 1
```

В данном примере при выполнении оператора **UPDATE** блок **SET** выполняется последовательно для каждой записи таблицы **SomeTable**. На каждом шаге значение переменной **@cnt** увеличивается на 1. В результате выполнения инструкции **UPDATE** получим в поле **RecNumber** каждой записи номер этой записи по порядку.

Во-вторых, появляется возможность выбирать данные из таблиц при помощи инструкции **UPDATE**:

```
DECLARE @firstName varchar(50),  
        @lastName varchar(70)  
  
UPDATE Clients  
SET @firstName = FirstName,  
    @lastName = LastName  
WHERE ClientID = 548
```

Такой прием удобно использовать в случае, если в момент выборки нужно установить на обрабатываемую запись эксклюзивную блокировку. Обычный оператор **SELECT** устанавливает разделяемую блокировку. Вопросы блокировок более подробно будут рассмотрены позже.

3. Удаление данных.

Для удаления записей из таблицы служит инструкция **DELETE**:

```
DELETE [FROM] tableName  
[FROM { tableName | vewName } [, { tableName | vewName } ...]]  
[WHERE searchCondition]
```

Как видно, синтаксис команды **DELETE** во многом аналогичен синтаксису команды **UPDATE**. Принципы использования тоже похожи.

Из таблицы удаляются все записи, удовлетворяющие условиям **WHERE**. В случае отсутствия блока **WHERE** удаляются все строки таблицы. При необходимости связки с другими таблицами используется предложение **FROM**.

Примеры.

```
-- Удаляем слишком старых клиентов  
DELETE FROM Clients  
WHERE DateBorn < '1900-01-01'  
  
-- Удаляем клиентов без счетов  
DELETE FROM Clients  
FROM Clients c  
WHERE NOT EXISTS( SELECT 1 FROM Accounts a WHERE a.ClientID = c.ClientID )  
  
-- Удаляем счета, открытые в той же валюте, что и счета клиента с ID 333  
DELETE FROM Accounts  
FROM Accounts a, Accounts a333  
WHERE a333.ClientID = 333  
    AND a.Currency = a333.Currency
```

4. Усекновение таблицы.

Зачастую быстро очистить таблицу. Такая операция выполняется при помощи инструкции

```
TRUNCATE TABLE tableName
```

После выполнения этой инструкции все данные из таблицы удаляются. При этом структура таблицы, допуски, ограничения, индексы и другие связанные с данной таблицей объекты сохраняются.

Данная команда является аналогом команды **DELETE** без условия **WHERE**, но выполняется гораздо быстрее. Однако отменить действие **TRUNCATE TABLE** нельзя: действие данной команды не проходит через лог транзакций.

TRUNCATE TABLE неприменима, если таблица-операнд имеет ограничения ссылочной целостности. Сначала нужно удалить строки из внешней таблицы или удалить внешние ключи.

TRUNCATE TABLE неприменима для таблиц, разбитых на разделы.

Пример:

```
TRUNCATE TABLE Accounts
```


5. Работа с данными типа text и image.

Типы **text** и **image** в ASE предназначены для хранения крупных блоков данных (объемом до 2GB). **text** ориентирован на хранение текстовой информации, **image** - бинарной.

Безусловно, работа с такими крупными блоками информации имеет свою специфику. Для манипулирования данными типа **text** и **image** предназначены следующие функции и команды.

Функция	Описание
patindex("%pattern%", charExpr [using {bytes chars characters}])	Возвращает позицию первого вхождения шаблона pattern в символьном выражении charExpr . Если шаблон не найден, возвращается 0 .
textptr(textColumnName)	Возвращает текстовое значение указателя, 16-байтовое значение varbinary .
textvalid("tableName.colName", textpointer)	Проверяет, целостность текстового указателя. Идентификатор для поля image/text , должен включать имя таблицы. Возвращается 1 , если указатель допустим, иначе - 0 .

Команда	Описание
writetext [[database.]owner.]tableName. columnName textPointer [readpast] [with log] data	Выполняет минимально логируемую модификацию существующего поля image/text .
readtext [[database.]owner.]tableName.columnName textPointer offset size	Позволяет получить определенную часть данных, хранимую в поле image/text . В качестве обязательных параметров требует имя таблицы и поля, текстовый указатель, начальное смещение внутри поля и размер данных в байтах или символах.

Данные типа **text/image** хранятся в связанном списке страниц, отдельно от самой таблицы.

Место под данные типа **text/image** не выделяется до инициализации (первой модификации или вставки непустого значения). Т.е. при явной или неявной вставке значения **NULL** текстовый указатель не создается. При инициализации распределяется минимум одна страница данных для каждого значения и полученный указатель помещается в основную таблицу.

Таким образом, минимальный объем данных, который занимает отличное от **NULL** значение поля **text/image**, - одна страница. Даже если полезной информации 1 байт.

По этой причине, для экономии дискового пространства, столбцы типа **text/image** обычно определяют допускающими значения **NULL**. При Update столбца значением **NULL**, освобождаются все распределенные ранее страницы, кроме первой которая остается доступной для будущего использования **writetext**. Чтобы освободить все занятые страницы, нужно удалить всю строку.

Тогда как явные операции вставки или модификации данных типа **text/image** проходят через лог транзакций полностью, операция **writetext** минимально логируема, т.е. в журнал транзакций заносится только информация о выделении или освобождении страниц данных.

Пример.

```
create table texttest (id varchar(6), huge_txt text null)
insert texttest (id, huge_txt) values ('7832', NULL)

select * from texttest
```

id	huge_txt
7832	NULL

```
declare @val varbinary(16)
select @val = textptr(huge_txt) from texttest where id = '7832'
writetext texttest.huge_txt @val with log 'hello world'
```

Server Message: Number 7133, Severity 16

Line 3:

NULL textptr passed to WRITETEXT function.

```
declare @val varbinary(16)
select @val = textptr(huge_txt) from texttest where id = '7832'
readtext texttest.huge_txt @val 1 5 using chars
```

Server Message: Number 7133, Severity 16

Line 3:

NULL textptr passed to READTEXT function.

```
insert texttest (id, huge_txt) values ('7833', 'Very big piece of text')
select * from texttest
```

id	huge_txt
7832	NULL
7833	Very big piece of text

```
declare @val varbinary(16)
select @val = textptr(huge_txt) from texttest where id = '7833'
writetext texttest.huge_txt @val with log 'Hello, world'
select * from texttest where id = '7833'
```

id	huge_txt
7833	Hello, world

```
declare @val varbinary(16)
select @val = textptr(huge_txt) from texttest where id = '7833'
readtext texttest.huge_txt @val 1 5 using chars
select @val as val
```

val
ello

6. Поля **timestamp**.

Как уже говорилось ранее, тип **timestamp** для ASE определен как **varbinary(16) null**.

Значения полей с типом **timestamp** заполняются сервером автоматически при любом изменении записи, содержащей это поле.

```
create table test( data varchar(20), stamp timestamp )

insert into test( data ) select 'First'
select * from test
```

data	stamp
First	30 30 30 30 30 30 30 30 34 31 37 36 31 42 32 41 37

```
update test set data = 'Second'
select * from test
```

data	stamp
Second	30 30 30 30 30 30 30 30 34 31 37 36 32 41 35 46 45

Как видно, несмотря на то, что и при вставке, и при изменении данных значение поля **stamp** явно не указывалось, его значение было сгенерировано и после изменения поля **data** также изменилось.

Sybase ASE гарантирует, что записи, измененный позднее, будут иметь большее значение поля **timestamp**.

Этим свойством удобно пользоваться для отслеживания изменений данных.

Пример 1. Копирование изменений.

Представим, что на од ном сервере есть таблица, в которую записываются оперативные данные. Записи в таблице могут добавляться и изменяться.

Копия этой таблицы хранится на другом сервере, который используется для построения отчетов. Синхронизация данных производится ежедневно, в дискретном режиме.

Возможный алгоритм организации процесса синхронизации, если в таблице-источнике присутствует поле **timestamp**:

Инициализация:

1. Выполняем копирование данных с источника на приемник, запоминаем максимальное обработанное значение **timestamp**.

Итерация:

1. Получаем максимальное значение `timestamp` из таблицы-истоника.
2. Производим копирование всех записей, у которых `timestamp` больше запомненного на предыдущей итерации и меньше либо равен текущему максимуму, полученному на шаге 1.
3. Запоминаем новое значение `timestamp`, полученное на шаге 1.
4. На следующей итерации повторяем шаги 1-3.

Пример 2. Проверка неизменности данных.

Обычный процесс редактирования информации в базе данных выглядит так:

1. Получаем информацию из БД
2. Заполняем полученной информацией форму редактирования
3. После редактирования сохраняем информацию в БД

Между пунктами 1 и 3 вполне может "вклиниться" другой процесс, который отредактирует ту же самую запись.

Блокировать запись на весь период редактирования неправильно, это может на неопределенный период помешать работе многих сотрудников.

Перезаписать то, что записал другой пользователь, тоже неправильно.

Если редактируемая запись содержит поле **timestamp**, можно модифицировать алгоритм следующим образом:

1. Получаем информацию из БД, запоминаем значение **timestamp**
2. Заполняем полученной информацией форму редактирования
3. После редактирования проверяем, совпадает ли текущее значение поля **timestamp** с запомненным на шаге 1
4. Если совпадает, сохраняем информацию в БД, иначе сообщаем пользователю о том, что кто-то успел раньше него